



Міністерство освіти і науки України
Національний технічний університет України
“Київський політехнічний інститут імені Ігоря Сікорського”
Факультет інформатики та обчислювальної техніки
Кафедра інформаційних систем та технологій

Лабораторна робота №4

з дисципліни «Технології розроблення програмного забезпечення»

Тема: «13. Office communicator»

Виконав:

студент групи ІА-32
Шадлер Андрій

Перевірив:

вик. Мягкий М. Ю.

Тема роботи

Використання патерна проєктування **Strategy (Стратегія)** у програмному забезпеченні.

Мета роботи

Ознайомитися з патерном проєктування **Strategy**, його призначенням та структурою, а також реалізувати даний патерн у власному програмному проєкті з метою підвищення гнучкості, розширюваності та підтримуваності коду.

Зміст

Теоретичні відомості	2
Хід виконання:	3
Завдання:	3
Діаграма класів:	3
Опис реалізації:	4
Вихідний код	5
Контрольні запитання	10
Висновки	13

Теоретичні відомості

Патерн проєктування Strategy

Strategy (Стратегія) – це поведінковий патерн проєктування, який дозволяє визначати сімейство алгоритмів, інкапсулювати кожен з них у окремому класі та робити ці алгоритми взаємозамінними.

Патерн Strategy дозволяє:

відокремити алгоритм від коду, який його використовує;

змінювати поведінку об'єкта під час виконання програми;

уникати великої кількості умовних операторів (if, switch);

дотримуватися принципу Open/Closed (відкритість для розширення, закритість для модифікації).

Структура патерна Strategy

Патерн Strategy складається з таких основних елементів:

Strategy – інтерфейс або абстрактний клас, що оголошує загальний метод алгоритму;

ConcreteStrategy – конкретні реалізації алгоритму;

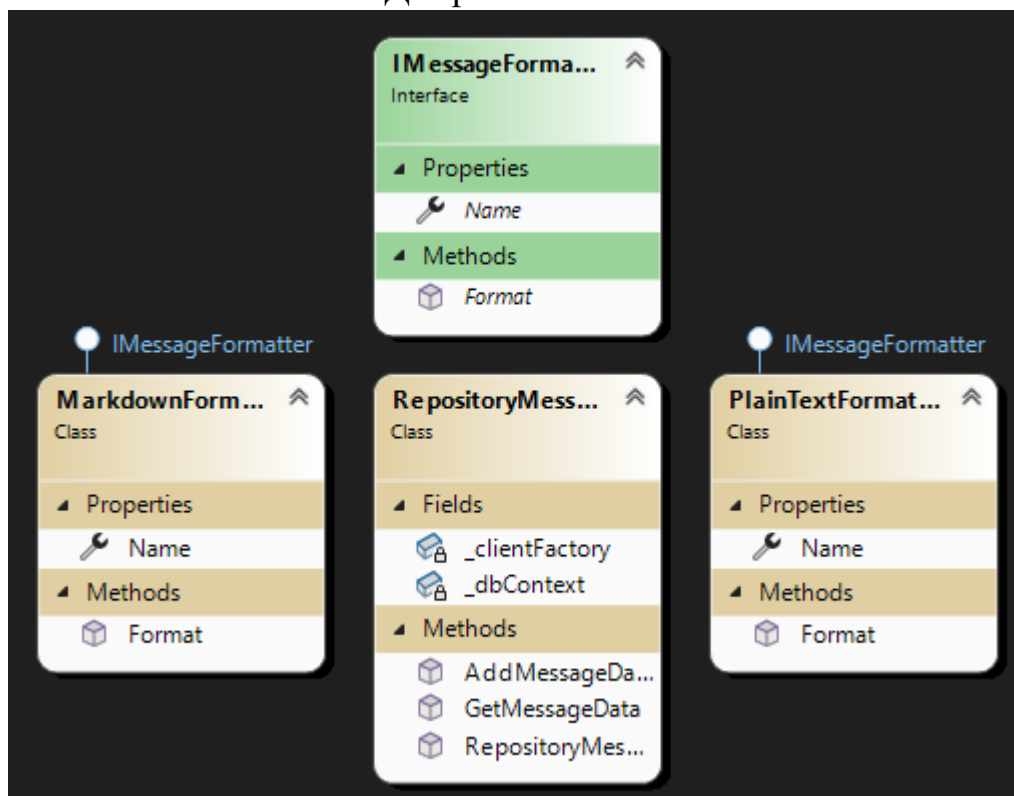
Context – клас, який використовує об'єкт стратегії та делегує йому виконання алгоритму.

Хід виконання:

Завдання:

- Ознайомитись з короткими теоретичними відомостями.
- Реалізувати частину функціоналу робочої програми у вигляді класів та їхньої взаємодії для досягнення конкретних функціональних можливостей.
- Реалізувати один з розглянутих шаблонів за обраною темою.
- Реалізувати не менше 3-х класів відповідно до обраної теми.
- Підготувати звіт щодо виконання лабораторної роботи. Поданий звіт повинен містити: діаграму класів, яка представляє використання шаблону в реалізації системи, навести фрагменти коду по реалізації цього шаблону.

Діаграма класів:



Опис реалізації:

Опис реалізації в лабораторній роботі

У даній лабораторній роботі патерн Strategy було реалізовано для задачі форматування тексту повідомлень у чаті.

Інтерфейс стратегії

Було створено інтерфейс:

`Services/Formatting/IMessageFormatter.cs`

Інтерфейс `IMessageFormatter` визначає контракт для форматування повідомлень та містить:

властивість `Name`, яка ідентифікує стратегію;

метод `Format(Message)`, що реалізує алгоритм форматування тексту.

Таким чином, інтерфейс `IMessageFormatter` виступає роллю Strategy.

Конкретні стратегії

Було реалізовано дві конкретні стратегії форматування:

`PlainTextFormatter` – повертає текст повідомлення без додаткового форматування;

`MarkdownFormatter` – застосовує форматування Markdown до тексту повідомлення.

Кожна стратегія інкапсулює власний алгоритм форматування та реалізує інтерфейс `IMessageFormatter`, що відповідає ролі ConcreteStrategy.

Контекст використання стратегії

Клас `RepositoryMessages` було модифіковано таким чином:

у конструктор додано ін'єкцію `IEnumerable<IMessageFormatter>`;

метод `GetMessageData` отримав додатковий необов'язковий параметр `formatName`;

після отримання даних з бази даних до тексту кожного повідомлення застосовується вибрана стратегія форматування.

Вибір конкретної стратегії здійснюється під час виконання програми на основі її імені (`Name`), що повністю відповідає ідеї патерна Strategy.

Реєстрація стратегій

У файлі Program.cs обидві реалізації форматування були зареєстровані в контейнері залежностей (DI), що дозволяє легко додавати нові стратегії без зміни існуючої логіки.

Відповідність патерну Strategy

Реалізація повністю відповідає патерну Strategy з таких причин:

Інкапсуляція поведінки – форматування винесене в окремі класи.

Наявність сімейства алгоритмів – кожен форматер реалізує власний алгоритм.

Взаємозамінність – конкретна стратегія обирається під час виконання програми.

Принцип Open/Closed – для додавання нового формату не потрібно змінювати код репозиторію.

Слабка зв'язаність та тестованість – RepositoryMessages залежить лише від абстракції IMessageFormatter.

Приклад використання

Отримання повідомлень із застосуванням конкретної стратегії форматування:

```
var messages = await _repositoryMessages.GetMessageData(chatId, "md");
```

У цьому випадку текст повідомлень буде відформатовано за допомогою MarkdownFormatter.

Вихідний код

```
// Services/Formatting/IMessageFormatter.cs
```

```
using Skype.Data;
```

```
namespace Skype.Services.Formatting
```

```
{
```

```

public interface IMessageFormatter
{
    string Name { get; }          // small discriminator for selection
    string Format(Message message);
}

// Services/Formatting/MarkdownFormatter.cs
using Skype.Data;
using System.Net;

namespace Skype.Services.Formatting
{
    public class MarkdownFormatter : IMessageFormatter
    {
        public string Name => "md";

        public string Format(Message message)
        {
            // basic markdown example – escape text
            var text = WebUtility.HtmlEncode(message.Text);
            return $"**{message.OwnerId}** _{message.TimeSend:HH:mm} _
\n{text}";
        }
    }
}

// Services/Formatting/PlainTextFormatter.cs
using Skype.Data;

```

```

namespace Skype.Services.Formatting
{
    public class PlainTextFormatter : IMessageFormatter
    {
        public string Name => "plain";

        public string Format(Message message)
        {
            // minimal, safe formatting
            return $"{message.TimeSend:yyyy-MM-dd HH:mm}
[ {message.OwnerId}]: {message.Text}";
        }
    }
}

```

```

using Microsoft.EntityFrameworkCore;
using Skype.Data;
using Skype.Services.Formatting;
using static Skype.DTOs.MessagesDTO;

```

```

namespace Skype.Repositories
{
    public class RepositoryMessages
    {
        AppDbContext _dbContext;
        private readonly IEnumerable<IMessageFormatter> _formatters;
    }
}

```

```

    public RepositoryMessages(AppDbContext dbContext,
        IEnumerable<IMessageFormatter> formatters)
    {
        _dbContext = dbContext;
        _formatters = formatters;
    }

    public async Task<List<MessageWithItsUserNameDTO>>
        GetMessageData(int chatId, string? formatName = null)
    {
        var list = await _dbContext.Messages
            .Where(m => m.ChatId == chatId)
            .Join(
                _dbContext.Users,
                message => message.OwnerId,
                user => user.Id,
                (message, user) => new MessageWithItsUserNameDTO
                {
                    Id = message.Id,
                    ChatId = message.ChatId,
                    OwnerId = message.OwnerId,
                    TimeSend = message.TimeSend,
                    Text = message.Text,

                    OwnerName = user.Name,
                }
            )
            .ToListAsync();
    }

```



```

        if (!string.IsNullOrEmpty(formatName))
        {
            var formatter = _formatters.FirstOrDefault(f => f.Name ==
formatName);
            if (formatter != null)
            {
                // Create a small Message instance for the formatter and replace
DTO text
                foreach (var dto in list)
                {
                    var msg = new Message
                    {
                        Id = dto.Id,
                        ChatId = dto.ChatId,
                        OwnerId = dto.OwnerId,
                        TimeSend = dto.TimeSend,
                        Text = dto.Text
                    };
                    dto.Text = formatter.Format(msg);
                }
            }
        }

        return list;
    }

```

```

public async Task AddMessageData(int chatId, int ownerId, string text)
{
    Message newMessage = new Message()

```

```

    {
        ChatId = chatId,
        OwnerId = ownerId,
        TimeSend = DateTime.UtcNow,
        Text = text
    };
    _dbContext.Messages.Add(newMessage);
    await _dbContext.SaveChangesAsync();
}
}
}

```

Контрольні запитання

1. Що таке шаблон проєктування?

Шаблон проєктування – це типовий, перевірений часом спосіб розв’язання поширеної проблеми проєктування програмного забезпечення в певному контексті. Він не є готовим кодом, а описує структуру та взаємодію класів.

2. Навіщо використовувати шаблони проєктування?

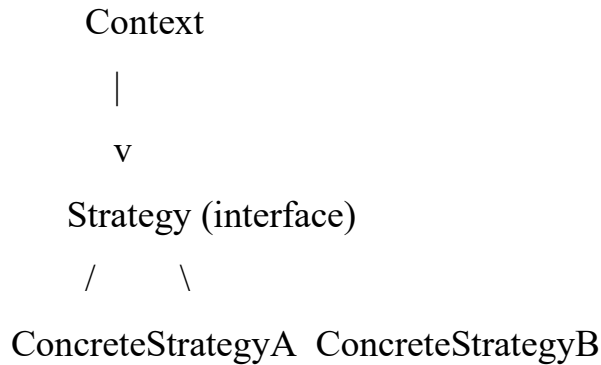
Шаблони проєктування використовуються для:

- підвищення гнучкості та розширюваності коду;
- зменшення зв’язаності між компонентами;
- повторного використання перевірених архітектурних рішень;
- полегшення підтримки та розуміння коду іншими розробниками.

3. Яке призначення шаблону «Стратегія»?

Шаблон **Strategy** (Стратегія) призначений для інкапсуляції сімейства алгоритмів і надання можливості змінювати алгоритм під час виконання програми незалежно від клієнтського коду.

4. Структура шаблону «Стратегія»



5. Класи шаблону «Стратегія» та їх взаємодія

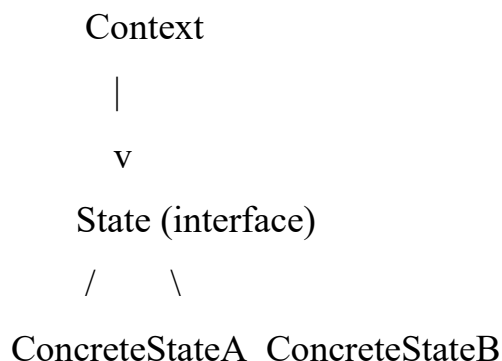
- **Strategy** – інтерфейс алгоритму.
- **ConcreteStrategy** – конкретні реалізації алгоритмів.
- **Context** – використовує об’єкт **Strategy** та делегує йому виконання алгоритму.

Context не знає деталей реалізації алгоритму.

6. Яке призначення шаблону «Стан»?

Шаблон **State (Стан)** дозволяє об’єкту змінювати свою поведінку залежно від внутрішнього стану, створюючи враження зміни класу об’єкта.

7. Структура шаблону «Стан»



8. Класи шаблону «Стан» та їх взаємодія

- **State** – інтерфейс для станів.
- **ConcreteState** – конкретні стани з власною поведінкою.

- **Context** – зберігає поточний стан і делегує йому поведінку.

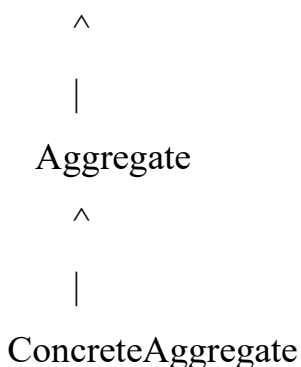
Стан може змінювати інший стан Context.

9. Яке призначення шаблону «Ітератор»?

Шаблон **Iterator** (Ітератор) забезпечує послідовний доступ до елементів колекції без розкриття її внутрішньої структури.

10. Структура шаблону «Ітератор»

Client -> Iterator <-> ConcreteIterator



11. Класи шаблону «Ітератор» та їх взаємодія

- **Iterator** – інтерфейс обходу елементів.
- **ConcreteIterator** – реалізує механізм обходу.
- **Aggregate** – інтерфейс колекції.
- **ConcreteAggregate** – конкретна колекція, яка створює ітератор.

12. В чому полягає ідея шаблону «Одинак»?

Шаблон **Singleton** (Одинак) гарантує існування лише одного екземпляра класу та надає глобальну точку доступу до нього.

13. Чому шаблон «Одинак» вважають «анти-шаблоном»?

Singleton вважають анти-шаблоном, оскільки він:

- створює приховані глобальні залежності;
- ускладнює тестування;

- порушує принцип єдиної відповідальності;
- часто використовується там, де краще застосувати Dependency Injection.

14. Яке призначення шаблону «Проксі»?

Шаблон **Proxy (Проксі)** надає об'єкт-замінник, який контролює доступ до реального об'єкта, додаючи додаткову логіку (кешування, перевірка доступу, відкладене створення).

15. Структура шаблону «Проксі»

Client

|

v

Subject (interface)

/ \

RealSubject Proxy

16. Класи шаблону «Проксі» та їх взаємодія

- **Subject** – спільний інтерфейс.
- **RealSubject** – реальний об'єкт.
- **Proxy** – зберігає посилання на RealSubject і контролює доступ до нього.
- **Client** – працює з об'єктом через Subject.

Висновки

У ході виконання лабораторної роботи було вивчено патерн проєктування Strategy та реалізовано його у програмному проєкті. Застосування патерна дозволило зробити код більш гнучким, розширюваним та зручним для супроводу. Реалізація дає змогу легко додавати нові алгоритми форматування без зміни існуючої бізнес-логіки, що підтверджує ефективність використання патерна Strategy.